



Docker

Docker first tiny steps

Erfan Taheri

What is Docker?

- ▶ Docker is a platform for building, running, and managing applications inside lightweight, portable containers.



Core terminology review

▶ Image

- ▶ A read-only template that defines what goes into a container (OS, libraries, app code). Built from a Dockerfile.

▶ Tag

- ▶ A label (version) attached to an image. Default is latest.

▶ Digest (Hash)

- ▶ A unique checksum (SHA256) identifying an image. Immutable.

▶ Layer

- ▶ Each step in a Dockerfile adds a new "layer" to an image (cached for efficiency).

▶ Container

- ▶ A running instance of an image. Isolated, lightweight, and portable.

Core terminology review

▶ Dockerfile

- ▶ A text file with instructions on how to build an image

▶ Docker Engine/Daemon

- ▶ The runtime that builds, runs, and manages Docker containers. Includes the daemon (dockerd) and CLI (docker).

▶ Docker CLI

- ▶ Command-line tool (docker) to interact with Docker Engine.

▶ Registry

- ▶ A storage/repository for Docker images (e.g., Docker Hub, private registries).

▶ Docker Hub

- ▶ The default public registry where most images are hosted.

Core terminology review (cont'd)

▶ Ephemeral Storage

- ▶ Temporary storage for container files. Exists only while the container runs; deleted when it stops.

▶ Copy-on-Write (CoW)

- ▶ Files are only copied into the writable layer if modified.

▶ Namespace

- ▶ Linux feature providing isolation (PID, net, IPC, mount, user, UTS).

▶ Cgroups (Control Groups)

- ▶ Kernel feature to limit & monitor resources (CPU, memory, I/O).

▶ Volume

- ▶ A persistent storage mechanism for containers (data survives even if container is deleted).

▶ Network

- ▶ A virtual network connecting containers together or with the outside world.

Docker editions

Docker comes in different editions depending on the use case:

▶ Docker Community Edition (CE)

- ▶ Free and open-source.
- ▶ For developers, hobbyists, small teams.
- ▶ Includes Docker Engine, CLI, and basic tools.
- ▶ Updated more frequently.

▶ Docker Enterprise Edition (EE) (later became Mirantis Container Runtime)

- ▶ Paid, enterprise-grade product.
- ▶ Added features: role-based access, advanced security, certified images, enterprise support.
- ▶ Targeted at production workloads.

▶ Docker Engine: The core runtime (daemon + CLI) that runs containers on Linux.

▶ Docker Desktop: A packaged app for Mac, Windows, and Linux, bundling

Installing Docker

Windows

macOS

Linux

- ▶ Use Docker Desktop (Windows/macOS) or Docker Engine (Linux).
- ▶ Requires 64-bit OS and virtualization enabled.
- ▶ Verify install: `docker --version` and `docker run hello-world`.

▶ Image Management

Command	Description	Example
<code>docker pull <image></code>	Download an image from a registry	<code>docker pull nginx:latest</code>
<code>docker images</code>	List all images on local machine	<code>docker images</code>
<code>docker rmi <image></code>	Remove an image	<code>docker rmi nginx:latest</code>
<code>docker build -t <name>:<tag> .</code>	Build an image from a Dockerfile	<code>docker build -t myapp:1.0 .</code>
<code>docker push <repo>/<image>:<tag></code>	Push image to registry	<code>docker push myrepo/myapp:1.0</code>

► Container Lifecycle

Command	Description	Example
<code>docker run [options] <image></code>	Start a new container	<code>docker run -d --name web nginx</code>
<code>docker start <container></code>	Start a stopped container	<code>docker start web</code>
<code>docker stop <container></code>	Stop a running container	<code>docker stop web</code>
<code>docker restart <container></code>	Restart a container	<code>docker restart web</code>
<code>docker rm <container></code>	Remove a container	<code>docker rm web</code>

► Monitoring & Interaction

Command	Description	Example
<code>docker ps</code>	List running containers	<code>docker ps</code>
<code>docker ps -a</code>	List all containers	<code>docker ps -a</code>
<code>docker logs <container></code>	Show container logs	<code>docker logs web</code>
<code>docker exec -it <container> <cmd></code>	Execute a command inside container	<code>docker exec -it web /bin/bash</code>

► Resource Management

Command	Description	Example
<code>docker stats</code>	Show live CPU, memory, network, and I/O usage of containers	<code>docker stats web</code>
<code>docker inspect <container></code>	Detailed info, including resource limits & usage	<code>docker inspect web</code>
<code>docker update --cpu <value> --memory <value> <container></code>	Dynamically adjust resource limits	<code>docker update --cpu 2 --memory 1g web</code>
<code>docker info</code>	Show Docker system-wide information, including resource availability	<code>docker info</code>
<code>docker system df</code>	Show disk usage by images, containers, volumes	<code>docker system df</code>

Resource constraints

▶ 1. CPU Management

- ▶ Use `--cpus` to limit available cores
- ▶ Assign specific CPU sets with `--cpuset-cpus`
- ▶ Prevent one container from starving others

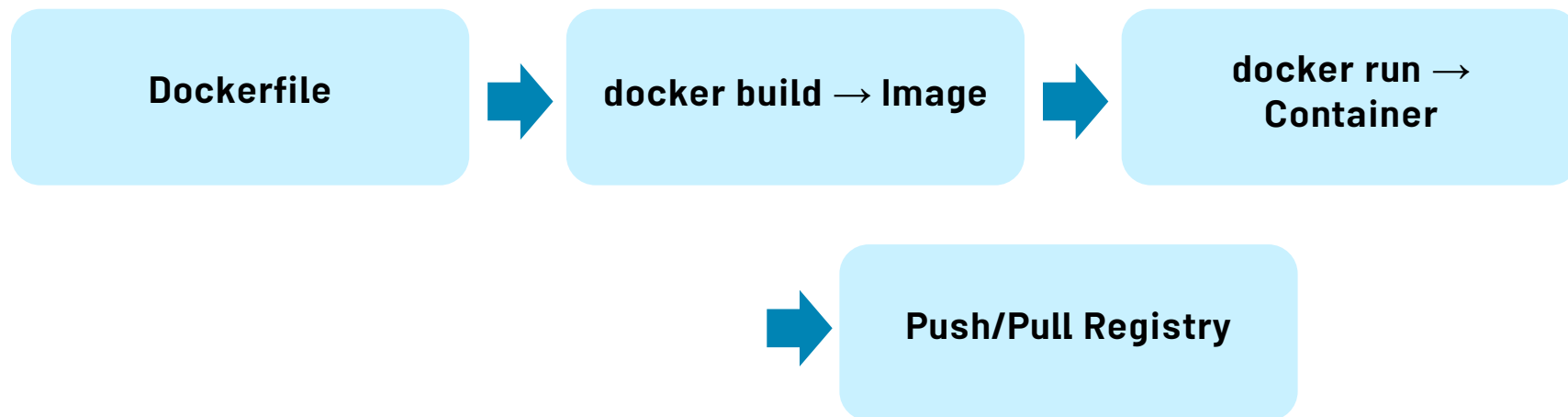
▶ 2. Memory Management

- ▶ Set hard limits with `-m / --memory`
- ▶ change OOM kill default behavior: `--oom-kill-disable`
- ▶ Define soft limits using `--memory-reservation`
- ▶ Control swap usage with `--memory-swap`

▶ 3. I/O & Network Control

- ▶ Block I/O: `--blkio-weight` for disk throughput fairness
- ▶ Network: Use `--network` to isolate & control bandwidth

Build & Run Workflow



What does `docker build` do?

- ▶ Takes a build context (directory) and Dockerfile to produce an image
- ▶ Executes Dockerfile instructions in order, producing read-only image layers
- ▶ Uses a storage driver (overlayfs, etc.) and build cache to reuse steps
- ▶ Build context is the directory sent to daemon - large context = slow builds
- ▶ Use .dockerignore to exclude node_modules, .git, build artifacts, secrets
- ▶ A Dockerfile contains all the commands to assemble an image
 - ▶ format: INSTRUCTION arguments

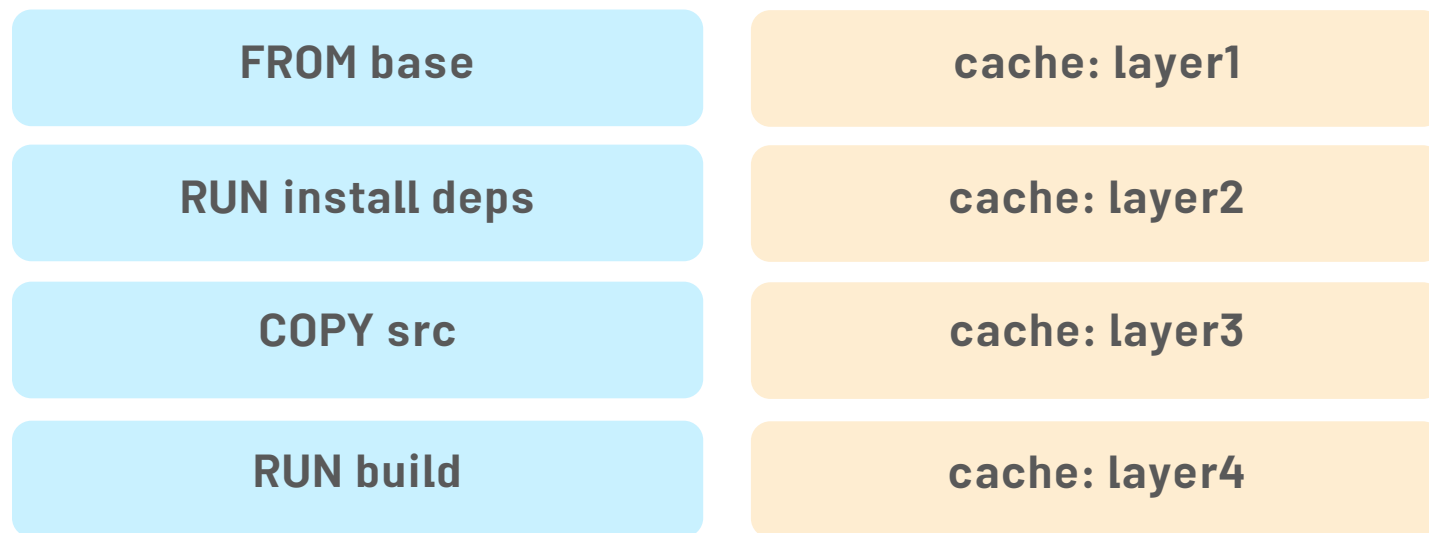
```
FROM python:3.12-alpine
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Instruction	Purpose	Cache impact	Example	Tips
FROM	Base image or stage	Resolves to image digest; pin for reproducibility	FROM node:20-alpine	Prefer digests for pinned builds
RUN	Execute commands (creates layer)	Changes contents → busts cache	RUN npm ci --silent	Group related commands; use --mount cache with BuildKit where helpful
COPY	Copy files from context	Any file change in copied path invalidates later steps	COPY package*.json ./	Copy manifests first to cache deps
ADD	Like COPY + auto-extract local tar + remote URL support	Same as COPY plus extraction effects	ADD https://example.com/file.tar.gz /	Prefer COPY unless you need URL or auto-extract
WORKDIR	Set working dir (auto-create)	Low impact	WORKDIR /app	Use early to keep RUN paths simple

instructions

Instruction	Purpose	Cache impact	Example	Tips
ENV	Set environment variables (persist)	Changes metadata; can affect runtime	ENV NODE_ENV=production	Use ARG for build-only values
ARG	Build-time variable (not persisted)	Affects build when used in instruction	ARG VERSION=1.0	Useful to bust cache intentionally
ENTRYPOINT/CMD	Define container process & defaults	No layer cost beyond metadata	ENTRYPOINT ["/app"] CMD ["--help"]	Use ENTRYPOINT for main binary, CMD for defaults
USER	Set user for subsequent commands & containers	No layer cost beyond metadata	USER app	Set in final stage to avoid root runtime
HEALTHCHECK	Define container health probe	Metadata; no large effect	HEALTHCHECK CMD curl -f /health exit 1	Keep lightweight checks

Layering & Build Cache

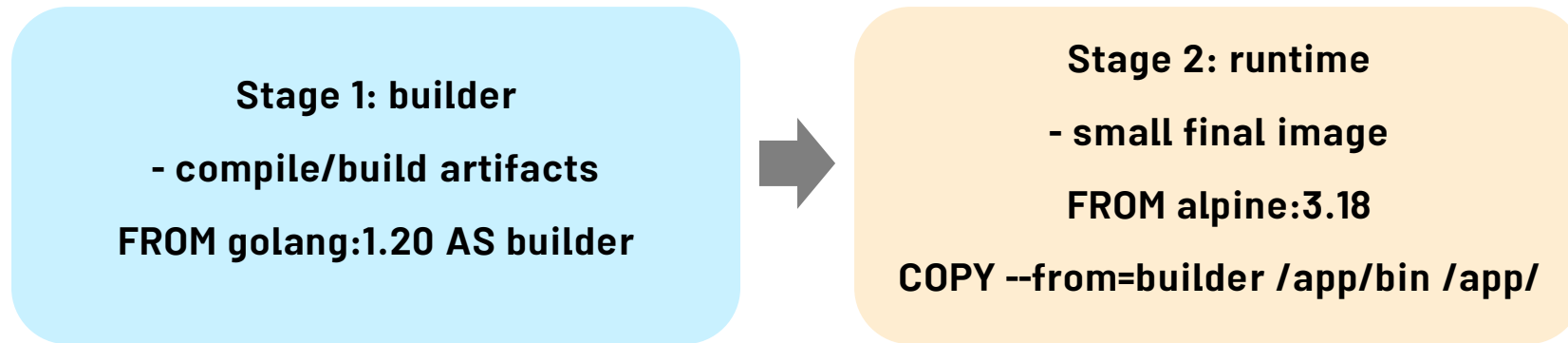


Cache hit → reuse layer
Cache miss → re-execute step

► Cache invalidation - common causes

- Any change to files used by a step invalidates that layer and subsequent ones
- COPY src/ . - changing any file in src/ busts cache for that step and after
- Best practice: install deps before copying source; leverage lockfiles to pin deps

Multi-stage Build (visual)



► Multi-stage reduces final image size by excluding build-time tools.

Example 1: multi-stage Dockerfile (node.js)

```
## Stage 1: builder
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --silent
COPY . .
RUN npm run build

## Stage 2: runtime
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
RUN npm prune --production
CMD ["node","dist/index.js"]
```

Example 2: multi-stage Dockerfile (maven)

```
# Stage 1: builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Runtime with lightweight JRE
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build Best Practices sum up

- ▶ **Prefer small base images (alpine/slim/distroless when appropriate)**
- ▶ **Use multi-stage builds to keep runtime images lean**
- ▶ **Pin versions and use reproducible builds**
- ▶ **Add a .dockerignore to reduce context size**
- ▶ **Run as non-root; drop unnecessary capabilities**
- ▶ **Leverage healthchecks and proper ENTRYPOINT/CMD**
- ▶ **Clean up build artifacts and package caches**

Docker Buildx

▶ BuildKit (2017)

- ▶ Introduced as a next-gen builder (separate project under Moby)
- ▶ Provided parallelized builds, better cache reuse, layer exports, and security features
- ▶ Became default backend in Docker 18.09+ (with DOCKER_BUILDKIT=1)

▶ Buildx (2019)

- ▶ CLI plugin to unlock full BuildKit capabilities
- ▶ Added multi-platform builds (via QEMU or remote builders)
- ▶ Enabled cache import/export, custom drivers, advanced outputs
- ▶ Became the standard way to build in CI/CD pipelines

▶ Today

- ▶ Buildx is the default builder in Docker Desktop
- ▶ Widely adopted in modern pipelines for speed, reproducibility, and security

▶ Multi-Platform Builds

- ▶ Build images for linux/amd64, linux/arm64, etc. in one command
- ▶ Leverages QEMU emulation or remote builders

▶ Advanced Caching

- ▶ Import/export cache (`--cache-from`, `--cache-to`)
- ▶ Cache mounts for package managers → faster, reproducible builds

▶ Security Features

- ▶ Secrets management (`--secret`) → don't bake secrets into images
- ▶ SSH forwarding for private repos (`--ssh`)

▶ Flexible Outputs

- ▶ Export as OCI layouts, tarballs, or push directly to registries
- ▶ Inline cache support for CI/CD pipelines

▶ Parallel & Efficient Builds

- ▶ Uses BuildKit under the hood → parallelized execution
- ▶ Optimizes layer reuse, reducing build time

Build Tooling Comparison

Capability	docker build	docker buildx build
Multi-platform (arm64/amd64)	No (single arch)	Yes
BuildKit features by default	Partial/legacy	Yes
Cache export/import	Limited	Yes (inline, registry, local)
Remote builders / drivers	No	Yes (docker-container, kubernetes)
Parallel stage builds	Limited	Yes
Reproducible builds	Manual effort	Improved support

Buildx Example

```
# syntax=docker/dockerfile:1.4
# Stage 1: Builder
FROM --platform=$BUILDPLATFORM golang:1.21-alpine AS builder
WORKDIR /app
COPY . .
RUN GOOS=$(echo $TARGETOS) GOARCH=$(echo $TARGETARCH) go build -o
myapp .

# Stage 2: Runtime
FROM alpine:3.18
WORKDIR /app
COPY --from=builder /app/myapp .
CMD ["/myapp"]
```

Buildx Example (cont'd)

```
$ docker buildx build \  
  --platform linux/amd64,linux/arm64 \  
  --cache-to=type=registry,ref=myrepo/myapp:cache,mode=max \  
  --cache-from=type=registry,ref=myrepo/myapp:cache \  
  -t myrepo/myapp:latest \  
  --push .
```